

Fundamentals of Programming Language 'C'

UNIT 2

Operators:

Operators are **symbols** used to **perform operations** on variables and values. For example: +, -, *, etc.

An **expression** is a combination of **variables, constants, and operators** that results in a value.

Arithmetic Operators

These operators perform **basic mathematical operations**.

Operator	Description	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	6 / 3	2
%	Modulus (Remainder)	5 % 2	1

Example

```
#include <stdio.h>
```

```
int main() {
```

```
    int a = 10, b = 3;
```

```
    printf("Addition: %d\n", a + b);    // 10 + 3 = 13
```

```
    printf("Subtraction: %d\n", a - b); // 10 - 3 = 7
```

```
    printf("Multiplication: %d\n", a * b); // 10 * 3 = 30
```

```
    printf("Division: %d\n", a / b);    // 10 / 3 = 3
```

```
    printf("Modulus: %d\n", a % b);    // 10 % 3 = 1
```

```
    return 0;

}
```

Output:

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3
Modulus: 1
```

Relational Operators

Used to **compare values**. They return **true (1)** or **false (0)**.

Operator	Description	Example
<code>==</code>	Equal to	<code>a == b</code>
<code>!=</code>	Not equal to	<code>a != b</code>
<code>></code>	Greater than	<code>a > b</code>
<code><</code>	Less than	<code>a < b</code>
<code>>=</code>	Greater or equal	<code>a >= b</code>
<code><=</code>	Less or equal	<code>a <= b</code>

Example:

```
#include <stdio.h>

int main() {

    int a = 10, b = 5;

    printf("a == b: %d\n", a == b);

    printf("a != b: %d\n", a != b);

    printf("a > b: %d\n", a > b);

    printf("a < b: %d\n", a < b);

    printf("a >= b: %d\n", a >= b);
```

```

printf("a <= b: %d\n", a <= b);

return 0;

}

```

Output:

```

a == b: 0
a != b: 1
a > b: 1
a < b: 0
a >= b: 1
a <= b: 0

```

Logical Operators

Used to combine relational expressions.

Operator	Name	Example
&&	Logical AND	a > 0 && b > 0
~		~
!	Logical NOT	! (a > b)

Example

```

#include <stdio.h>

int main() {

    int a = 5, b = 0;

    printf("a && b: %d\n", a && b);

    printf("a || b: %d\n", a || b);

    printf("!a: %d\n", !a);

    return 0;

}

```

Output:

```
a && b: 0  
a || b: 1  
!a: 0
```

Assignment Operators

Used to **assign values** to variables.

Operator	Example	Meaning
<code>=</code>	<code>a = 5</code>	Assign 5 to a
<code>+=</code>	<code>a += 3</code>	<code>a = a + 3</code>
<code>-=</code>	<code>a -= 3</code>	<code>a = a - 3</code>
<code>*=</code>	<code>a *= 3</code>	<code>a = a * 3</code>
<code>/=</code>	<code>a /= 3</code>	<code>a = a / 3</code>

Example:

```
#include <stdio.h>  
  
int main() {  
    int a = 10;  
  
    a += 5; // a = 15  
  
    a -= 3; // a = 12  
  
    a *= 2; // a = 24  
  
    a /= 4; // a = 6  
  
    a %= 4; // a = 2  
  
    printf("Final value of a: %d\n", a);  
  
    return 0;  
}
```

Output:

Final value of a: 2

Increment and Decrement Operators

Used to **increase or decrease** a variable by 1.

Operator	Name	Example	Meaning
<code>++</code>	Increment	<code>a++</code>	<code>a = a + 1</code>
<code>--</code>	Decrement	<code>a--</code>	<code>a = a - 1</code>

Example

```
#include <stdio.h>

int main() {

    int a = 5;

    printf("Post-increment: %d\n", a++); // a = 6 after print

    printf("Pre-increment: %d\n", ++a); // a = 7 before print

    printf("Post-decrement: %d\n", a--); // a = 6 after print

    printf("Pre-decrement: %d\n", --a); // a = 5 before print

    return 0;

}
```

Output:

Post-increment: 5
Pre-increment: 7
Post-decrement: 7
Pre-decrement: 5

Conditional Operator (Ternary Operator)

Short form of **if-else**.

Syntax:

```
(condition) ? true_result : false_result;
```

Example:

```
#include <stdio.h>

int main() {

    int a = 10, b = 20;

    int max = (a > b) ? a : b;

    printf("Maximum = %d\n", max);

    return 0;

}
```

Output:

Maximum = 20

Bitwise Operators

Used for **bit-level operations**

Operator	Description	Example
<code>&</code>	AND	<code>a & b</code>
<code> </code>	OR	<code>a b</code>
<code>^</code>	XOR	<code>a ^ b</code>
<code>~</code>	NOT (One's comp.)	<code>~a</code>
<code><<</code>	Left Shift	<code>a << 1</code>
<code>>></code>	Right Shift	<code>a >> 1</code>

Example

```
#include <stdio.h>

int main() {

    int a = 5, b = 3;

    printf("a & b = %d\n", a & b); // 1

    printf("a | b = %d\n", a | b); // 7

    printf("a ^ b = %d\n", a ^ b); // 6

    printf("~a = %d\n", ~a);        // -6 (1's complement)

    printf("a << 1 = %d\n", a << 1); // 10

    printf("a >> 1 = %d\n", a >> 1); // 2

    return 0;

}
```

Output:

```
a & b = 1
a | b = 7
a ^ b = 6
~a = -6
a << 1 = 10
a >> 1 = 2
```

Special Operators

Operator	Description	Example
<code>sizeof</code>	Returns size of data type/variable	<code>sizeof(int)</code>
<code>&</code>	Address of variable	<code>&a</code>
<code>*</code>	Pointer (value at address)	<code>*ptr</code>
<code>-></code>	Access struct member via pointer	<code>ptr->rollNo</code>

Example

```
#include <stdio.h>

int main() {

    int a = 10;

    int *p = &a;

    printf("Address of a: %p\n", &a);

    printf("Value using pointer: %d\n", *p);

    printf("Size of int: %lu\n", sizeof(a));

    return 0;

}
```

Output:

```
Address of a: 0x7ffee3c4e6fc
Value using pointer: 10
Size of int: 4
```

Arithmetic Expressions

Expressions that include **arithmetic operators**.

Example:

```
int a = 5, b = 2;
int result = a * b + 10; // result = 5*2 + 10 = 20
```

Evaluation of Expressions

Expressions are evaluated based on **operator precedence** and **associativity**.

Multiplication and division are evaluated **before** addition or subtraction.

Precedence of Arithmetic Operators

Precedence	Operators	Associativity
1 (highest)	<code>*</code> , <code>/</code> , <code>%</code>	Left to Right
2	<code>+</code> , <code>-</code>	Left to Right

Example with Precedence:

```
int x = 5 + 3 * 2; // x = 5 + 6 = 11 (multiplication first)
```

Type Conversions in Expressions

What is Type Conversion?

When operands in an expression are of **different data types**, C automatically converts them to the same type to perform the operation. This is called **type conversion**.

Types of Type Conversions:

a) Implicit Type Conversion (Automatic)

C automatically converts the **lower data type to higher data type** to prevent data loss.

Lower Type → Higher Type

char → int

int → float

float → double

Example:

```
#include <stdio.h>
```

```
int main() {
    int a = 5;
    float b = 2.5;
    float result = a + b; // 'a' is converted to float automatically

    printf("Result = %.2f\n", result);
    return 0;
}
```

Output:

Result = 7.50

b) Explicit Type Conversion (Type Casting)

Manual conversion using syntax:

(type) expression

Example:

```
#include <stdio.h>
```

```
int main() {  
    int a = 5, b = 2;  
    float result = (float)a / b; // a is explicitly cast to float  
  
    printf("Result = %.2f\n", result);  
    return 0;  
}
```

Output:

Result = 2.50

Operator Precedence and Associativity

What is Precedence?

Precedence determines which operator is evaluated first in an expression with multiple operators.

What is Associativity?

When two operators of same precedence appear, associativity decides the order of evaluation (left to right or right to left).

Precedence and Associativity Table (Important Operators):

Operator(s)	Description	Associativity
()	Function call	Left to Right
++, --	Increment/Decrement	Right to Left
*, /, %	Multiplication/Division	Left to Right
+, -	Addition/Subtraction	Left to Right

Operator(s)	Description	Associativity
<, >, <=, >=	Relational Operators	Left to Right
==, !=	Equality Operators	Left to Right
&&	Logical AND	Left to Right
,		,
=, +=, -= etc.	Assignment Operators	Right to Left
?:	Conditional Operator	Right to Left

Example:

```
#include <stdio.h>
int main() {
    int a = 2, b = 3, c = 4;
    int result = a + b * c;

    printf("Result = %d\n", result);
    return 0;
}
```

Output:

Result = 14

Mathematical Functions in C

Header File:

To use math functions, include:

```
#include <math.h>
```

Common Math Functions:

Function	Description	Example
<code>sqrt(x)</code>	Square root	<code>sqrt(16) → 4.0</code>
<code>pow(x, y)</code>	x raised to the power y	<code>pow(2, 3) → 8.0</code>
<code>abs(x)</code>	Absolute value of integer	<code>abs(-5) → 5</code>
<code>fabs(x)</code>	Absolute value of float	<code>fabs(-2.5) → 2.5</code>
<code>ceil(x)</code>	Round up	<code>ceil(2.3) → 3.0</code>
<code>floor(x)</code>	Round down	<code>floor(2.9) → 2.0</code>

Function	Description	Example
<code>log(x)</code>	Natural log (base e)	<code>log(1) → 0.0</code>
<code>log10(x)</code>	Log base 10	<code>log10(100) → 2.0</code>
<code>sin(x)</code>	Sine (x in radians)	<code>sin(0) → 0.0</code>
<code>cos(x)</code>	Cosine	<code>cos(0) → 1.0</code>

Example:

```
#include <stdio.h>
#include <math.h>
```

```
int main() {
    double a = 16.0, b = -3.7;

    printf("Square root: %.2f\n", sqrt(a));
    printf("Power (2^3): %.2f\n", pow(2, 3));
    printf("Ceil: %.2f\n", ceil(b));
    printf("Floor: %.2f\n", floor(b));
    printf("Absolute: %.2f\n", fabs(b));

    return 0;
}
```

Output:

```
Square root: 4.00
Power (2^3): 8.00
Ceil: -3.00
Floor: -4.00
Absolute: 3.70
```

Input & Output in C (I/O Operators)

C language uses **standard functions** for input and output. These functions are defined in the **stdio.h** (Standard Input Output Header File).

To use input/output in a C program, include this at the top:

```
#include <stdio.h>
```

1. Reading a Character

Function: getchar()

- Used to **read a single character** from the keyboard.
- No parameters.

Example:

```
#include <stdio.h>

int main() {

    char ch;

    printf("Enter a character: ");

    ch = getchar(); // reads a character

    printf("You entered: ");

    putchar(ch);    // displays the character

    return 0;

}
```

Output:

```
Enter a character: A
You entered: A
```

2. Writing a Character

Function: putchar()

- Used to **display a single character** on the screen.

3. Formatted Input

Function: scanf()

- Used to **take input** from the user.
- Allows input of various data types using **format specifiers**.

Syntax:

```
scanf("format_specifier", &variable);
```

◆ Common Format Specifiers:

Data Type	Format Specifier
int	%d
float	%f
char	%c
double	%lf
string	%s

Example:

```
#include <stdio.h>

int main() {

    int age;

    float height;

    printf("Enter age and height: ");

    scanf("%d %f", &age, &height); // formatted input

    printf("Age = %d, Height = %.2f\n", age, height);

    return 0;

}
```

Output:

```
Enter your age and height: 20 5.8
Age: 20
Height: 5.80
```

4. Formatted Output

Function: printf()

- Displays output on the screen.
- Can be used to show **text, variables, results**, etc.
- Supports format specifiers.

Syntax:

```
printf("text or format string", variables);
```

Example:

```
#include <stdio.h>

int main() {
    int roll = 101;
    float marks = 88.65;

    printf("Roll Number: %d\n", roll);
    printf("Marks: %.2f\n", marks);

    return 0;
}
```

Output:

```
Roll Number: 101
Marks: 88.65
```